Suma de prefijos
Fenwick Tree
Segment Tree

Que son la suma de prefijos, Fenwick Tree Segment Tree?

- Son **estructuras de datos** para organizar y almacenar datos de manera que podamos realizar consultas **(queries)** y modificaciones de manera **eficiente.**

00

Conceptos Bases

Propiedad Asociativa

Un operador binario $\oplus$ se dice asociativo, si siempre se cumple que:

$$((a \oplus b) \oplus c) = (a \oplus (b \oplus c))$$

Ejemplos

$$(1 + 2) + 3 = 1 + (2 + 3)$$

$$(1 \times 2) \times 3 = 1 \times (2 \times 3)$$

$$(1 \text{ xor } 2) \text{ xor } 3 = 1 \text{ xor } (2 \text{ xor } 3)$$

$$\text{MAX}(\text{MAX}(1,2),3) = \text{MAX}(1,\text{MAX}(1,2))$$

$$\text{MIN}(\text{MIN}(1,2),3) = \text{MIN}(1,\text{MIN}(1,2))$$

$$(\text{“A”} + \text{“B”}) + \text{“C”} = \text{“A”} + (\text{“B”} + \text{“C”})$$

Elemento Neutro

Un elemento neutro para un operador $\oplus$ es un **e** con la propiedad de que para cualquier x siempre se cumple

$$x \oplus e = e \oplus x = x$$

Ejemplo Elemento Neutro

$$1 + 0 = 1$$

$$10 \times 1 = 10$$

$$2 \text{ xor } 0 = 2$$

$$\text{MAX}(2, -\infty) = 2$$

$$\text{MIN}(5, +\infty) = 5$$

“My String” + “” = “My String”

Ejemplo Query

Consulta la suma del rango **2 a 5** del arreglo **arr**

$$\sum_{i=2}^{5} \texttt{arr}[i] = 6 + 4 + 2 + 5 = 17.$$

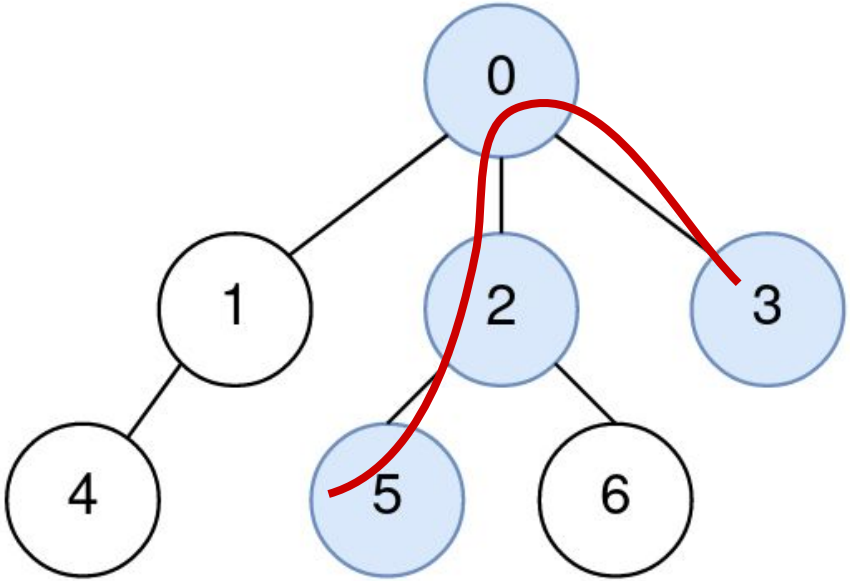
Index i	1	2	3	4	5	6
$\texttt{arr}[i]$	1	6	4	2	5	3

Ejemplo Query 2D

0	0	0	0	0	0
0	1	5	6	11	8
0	1	7	11	9	4
0	4	6	1	3	2
0	7	5	4	2	3

Ejemplo Query Árbol

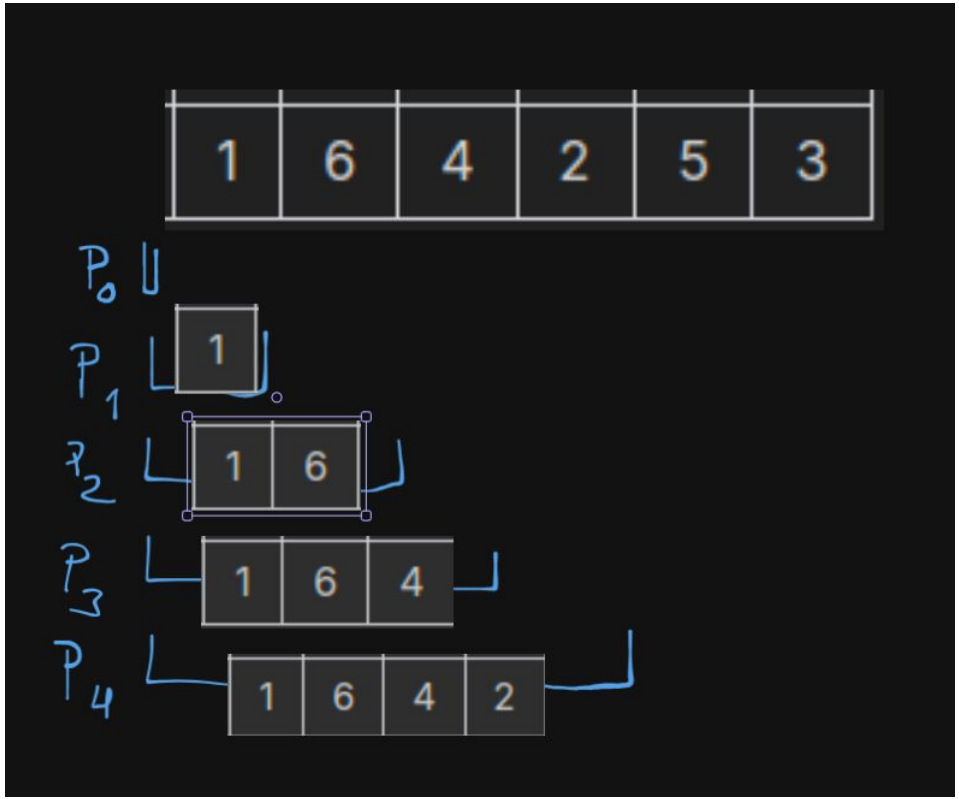
Suma del path del nodo 5 al 3



01

Suma de Prefijos

Prefijos



Suma Prefijos

1	6	4	2	5	3

P_0 []

P_1 [1]

P_2 [1 + 6]

P_3 [1 + 6 + 4]

P_4 [1 + 6 + 4 + 2]

$$\texttt{prefix}[k] = \sum_{i=1}^{k} \texttt{arr}[i]$$

$$\texttt{prefix}[k] = \texttt{prefix}[k-1] + \texttt{arr}[k]$$

Suma de Prefijos

- Dado un rango de un arreglo L a R, devolver su suma.

$$\sum_{i=L}^{R} \mathtt{arr}[i]$$

Suma de Prefijos

- Dado un rango de un arreglo L a R, devolver su suma.

$$\sum_{i=L}^{R} \mathtt{arr}[i] = \sum_{i=1}^{R} \mathtt{arr}[i] - \sum_{i=1}^{L-1} \mathtt{arr}[i]$$

Suma de Prefijos

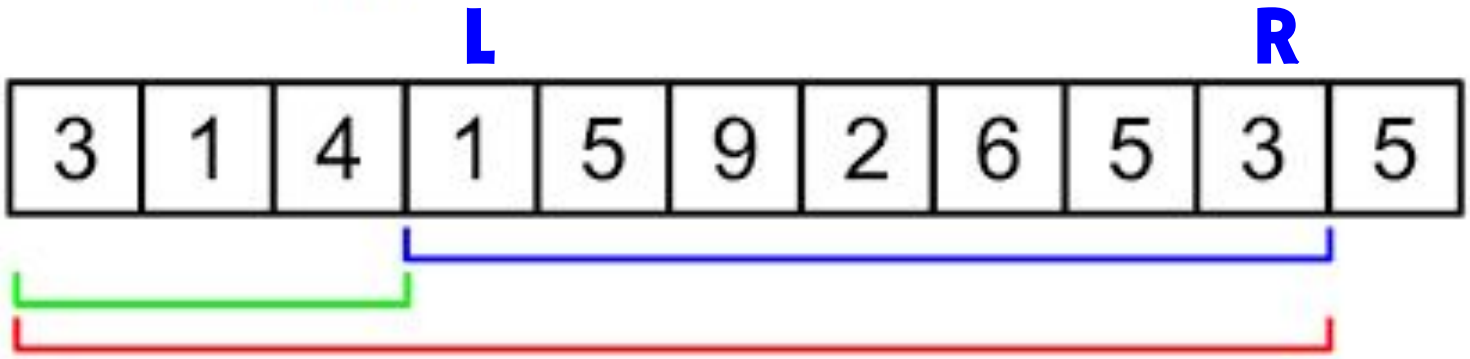
- Dado un rango de un arreglo L a R, devolver su suma.

$$\sum_{i=L}^{R} \texttt{arr}[i] = \sum_{i=1}^{R} \texttt{arr}[i] - \sum_{i=1}^{L-1} \texttt{arr}[i]$$

$$\sum_{i=L}^{R} \texttt{arr}[i] = \texttt{prefix}[R] - \texttt{prefix}[L-1]$$

Suma de Prefijos

$$\sum_{i=L}^{R} \texttt{arr}[i] = \texttt{prefix}[R] - \texttt{prefix}[L-1]$$



Sum of blue interval = 39 - 8 = 31

Código Suma de Prefijos

https://usaco.guide/silver/prefix-sums?lang=cpp#solution---static-range-sum

Calcular Suma:

```
vl psum(const vl &a) {
    vl psum(sz(a) + 1);
    for (int i = 0; i < sz(a); ++i) psum[i + 1] = psum[i] + a[i];
    // or partial_sum(begin(a),end(a),begin(psum)+1);
    return psum;
}
```

Responder query [L, R] (índice base 0):

```
for (int i = 0; i < Q; ++i) {
    int l, r;
    cin >> l >> r;
    cout << p[r+1] - p[l] << "\n";
}
```

No puedes hacer updates con Prefix Sum!

Tiempo de Complejidad

Crear prefijos: O(n)

Responder Query: O(1)

Problema 1

Given an array of n integers, your task is to count the number of subarrays having sum x.

Constraints

- $1 \leq n \leq 2 \cdot 10^5$
- $-10^9 \leq x, a_i \leq 10^9$

Example

Input:

```
5 7
2 -1 3 5 -2
```

Output:

```
2
```

Problema 2

Given an array of n integers, your task is to count the number of subarrays where the sum of values is divisible by n.

Constraints

- $1 \leq n \leq 2 \cdot 10^5$
- $-10^9 \leq a_i \leq 10^9$

Example

Input:
5
3 1 2 7 4

Output:
1

Problema 3 (Tarea)

You are given an array $a_1, a_2, \ldots, a_n$ consisting of integers from 0 to 9. A subarray $a_l, a_{l+1}, a_{l+2}, \ldots, a_{r-1}, a_r$ is good if the sum of elements of this subarray is equal to the length of this subarray ($\sum_{i=l}^{r} a_i = r - l + 1$).

For example, if $a = [1, 2, 0]$, then there are 3 good subarrays: $a_{1\ldots1} = [1], a_{2\ldots3} = [2, 0]$ and $a_{1\ldots3} = [1, 2, 0]$.

Calculate the number of good subarrays of the array a.

$(1 \leq n \leq 10^5)$ — the length of the array a.

Problema 4

You are given an $n \times n$ grid representing the map of a forest. Each square is either empty or contains a tree. The upper-left square has coordinates $(1, 1)$, and the lower-right square has coordinates (n, n).

Your task is to process q queries of the form: how many trees are inside a given rectangle in the forest?

Constraints

- $1 \leq n \leq 1000$
- $1 \leq q \leq 2 \cdot 10^5$
- $1 \leq y_1 \leq y_2 \leq n$
- $1 \leq x_1 \leq x_2 \leq n$

Input:

```
4 3
.*..
*.**
**..
****
2 2 3 4
3 1 3 1
1 1 2 2
```

Output:

```
3
1
2
```

Prefix Sum 2D

Calcular Prefix Sum 2D:

$$\texttt{prefix}[a][b] = \sum_{i=1}^{a} \sum_{j=1}^{b} \texttt{arr}[i][j].$$

$$\begin{aligned}\texttt{prefix}[i][j] = \texttt{prefix}[i-1][j] + \texttt{prefix}[i][j-1] \\ - \texttt{prefix}[i-1][j-1] + \texttt{arr}[i][j]\end{aligned}$$

Responder Query:

$$\begin{aligned}\sum_{i=a}^{A} \sum_{j=b}^{B} \texttt{arr}[i][j] = \texttt{prefix}[A][B] - \texttt{prefix}[a-1][B] \\ - \texttt{prefix}[A][b-1] + \texttt{prefix}[a-1][b-1]\end{aligned}$$

Prefix Sum 2D

we have a submatrix sum of $7 + 11 + 9 + 6 + 1 + 3 = 37$.

0	0	0	0	0	0
0	1	5	6	11	8
0	1	7	11	9	4
0	4	6	1	3	2
0	7	5	4	2	3

Código Prefix Sum

https://usaco.guide/silver/more-prefix-sums?lang=cpp#solution---forest-queries

```
vector<vl> matrix(n,vl(n)); // leer entrada
vector<vl> pf(n+1,vl(n+1));
for (int i =0;i<n;i++) { // calcular Prefix
    for (int j = 0;j<n;j++) {
        pf[i+1][j+1] = matrix[i][j] + pf[i][j+1] + pf[i+1][j] - pf[i][j];
    }
}
while (q--) { // Responder Query
    ll y1,x1,y2,x2;
    cin >>x1>>y1>>x2>>y2;
    y1--,x1--,y2--,x2--;
    ll sum = pf[x2+1][y2+1]-pf[x1][y2+1]-pf[x2+1][y1]+pf[x1][y1];
    cout << sum << "\n";
}
```

Tiempo de Complejidad

Crear prefijos: O(n*m)

Responder Query: O(1)

02

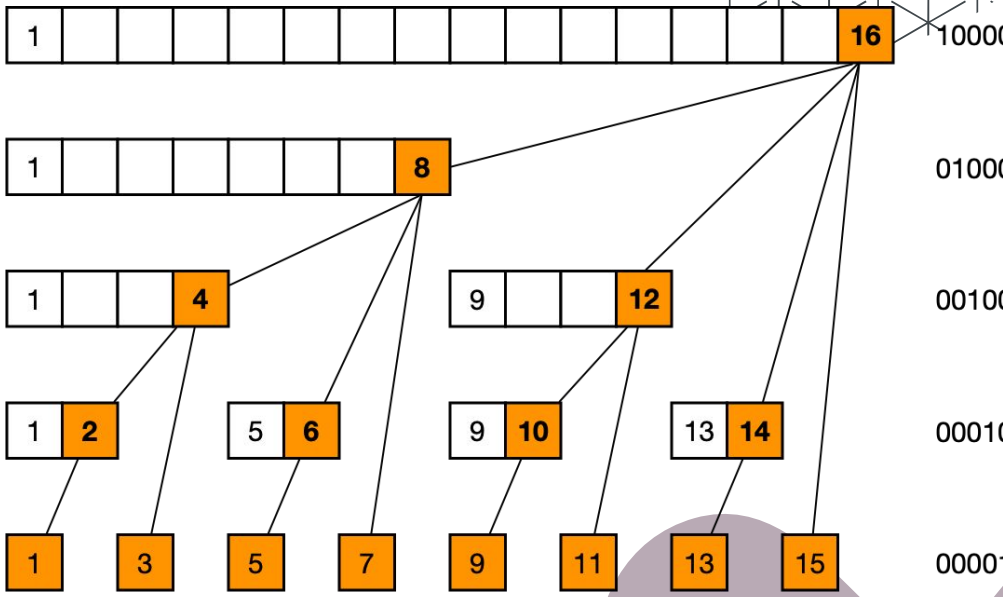
Fenwick Tree (BIT)

Lo mismo que Prefix Sum pero con Updates

Fenwick Tree - Binary Indexed Tree (BIT)

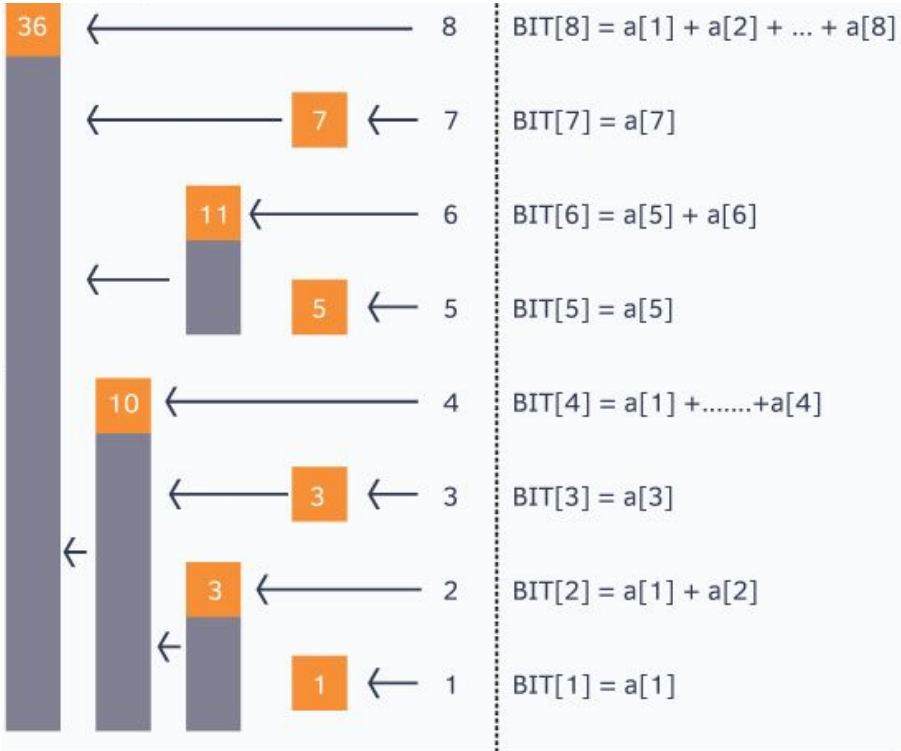
- Se representa como un array donde cada índice almacena información sobre sumas de segmentos del array original.

- Utiliza propiedades de números binarios para determinar los índices afectados durante actualizaciones y consultas.



Fenwick Tree - Binary Indexed Tree (BIT)

```
int a[] = {0, 1, 2, 3, 4, 5, 6, 7, 8}
```



Código

```
struct BIT { // 1-indexed, your first element of the array is at index 1
    vl bit;
    ll n;
    BIT(int n) : bit(n+1), n(n) {}

    ll lsb(int i) { return i & -i; } // least significant bit

    void add(int i, ll x) {
        for (; i <= n; i += lsb(i)) bit[i] += x;
    }

    ll sum(int r) {
        ll res = 0;
        for (; r > 0; r -= lsb(r)) res += bit[r];
        return res;
    }

    ll sum(int l, int r) {
        return sum(r) - sum(l-1);
    }

    void set(int i, ll x) {
        add(i, x - sum(i, i));
    }
};
```

Código

Ver Structs FenwickTreeOneBasedIndexing y FenwickTree:

https://cp-algorithms.com/data_structures/fenwick.html

Tiempo de Complejidad

Crear BIT: O(n * log (n))

Responder Query: O(log(n))

Actualizar: O(log(n))

Código 2D

```
struct BIT2D { // 1-indexed
    vector<vl> bit;
    ll n, m;
    BIT2D(ll n, ll m) : bit(n+1, vl(m+1)), n(n), m(m) {}

    ll lsb(ll i) { return i & -i; }

    void add(int row, int col, ll x) {
        for (int i = row;i<=n;i+=lsb(i))
            for (int j = col;j<=m;j+=lsb(j))
                bit[i][j] += x;
    }

    ll sum(int row, int col) {
        ll res = 0;
        for (int i = row;i>0;i-=lsb(i))
            for (int j = col;j>0;j-=lsb(j))
                res += bit[i][j];
        return res;
    }

    ll sum(int x1, int y1, int x2, int y2) {
        return sum(x2,y2) - sum(x1-1,y2) - sum(x2,y1-1) + sum(x1-1,y1-1);
    }

    void set(int x, int y, ll val) {
        add(x,y,val-sum(x,y,x,y));
    }
};
```

Tiempo de Complejidad

Crear BIT: O(n*m * log (n)*log(m))

Responder Query: O(log(n)*log(m))

Actualizar: O(log(n)*log(m))

Ejercicio

Given an array of n integers, your task is to process q queries of the following types:

1. update the value at position k to u
2. what is the sum of values in range $[a, b]$?

Ejercicio 2

You are given an $n \times n$ grid representing the map of a forest. Each square is either empty or has a tree. Your task is to process q queries of the following types:

1. Change the state (empty/tree) of a square.
2. How many trees are inside a rectangle in the forest?

03

Segment Tree

El más poderoso de los 3

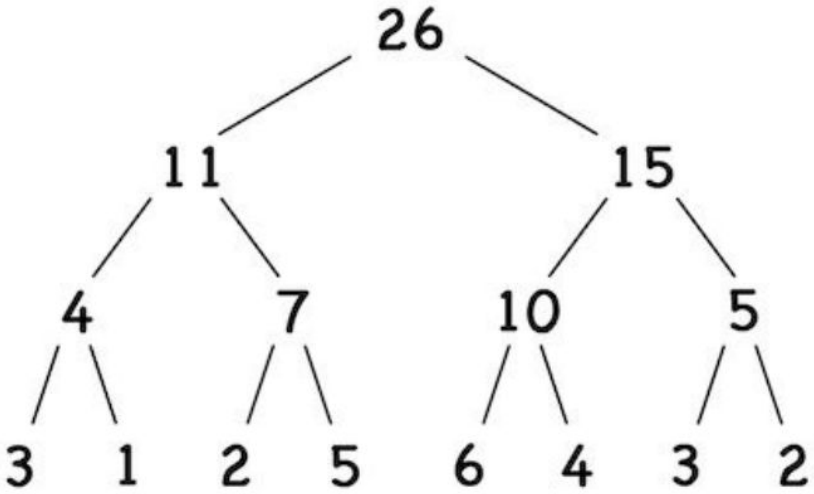
Segment Tree

Es un Árbol Binario perfecto, donde cada nodo guarda información de un intervalo [L, R]

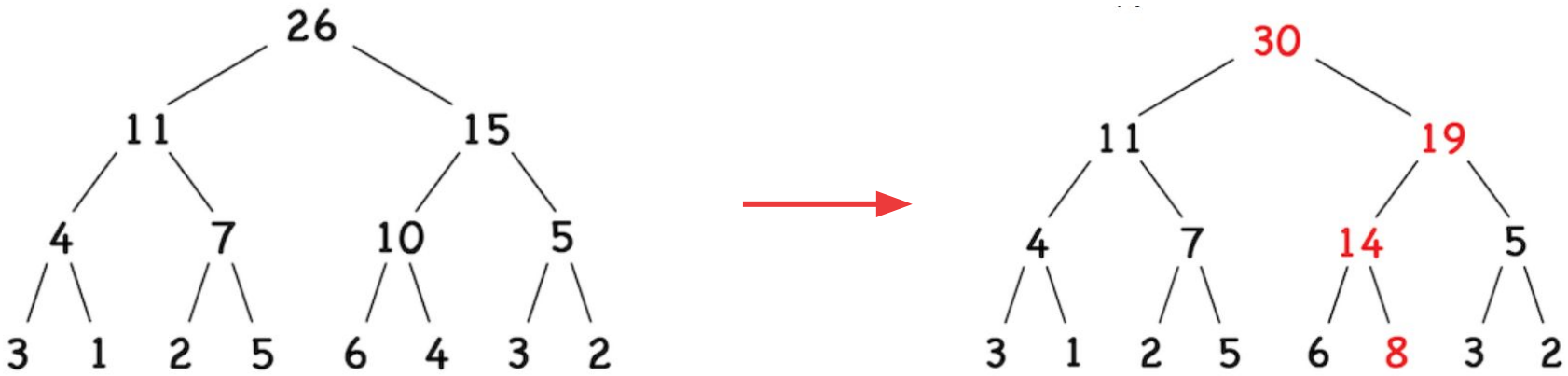
Segment Tree

Es un Árbol Binario perfecto, donde cada nodo guarda información de un intervalo [L, R]

A = {3, 1, 2, 5, 6, 4, 3, 2}

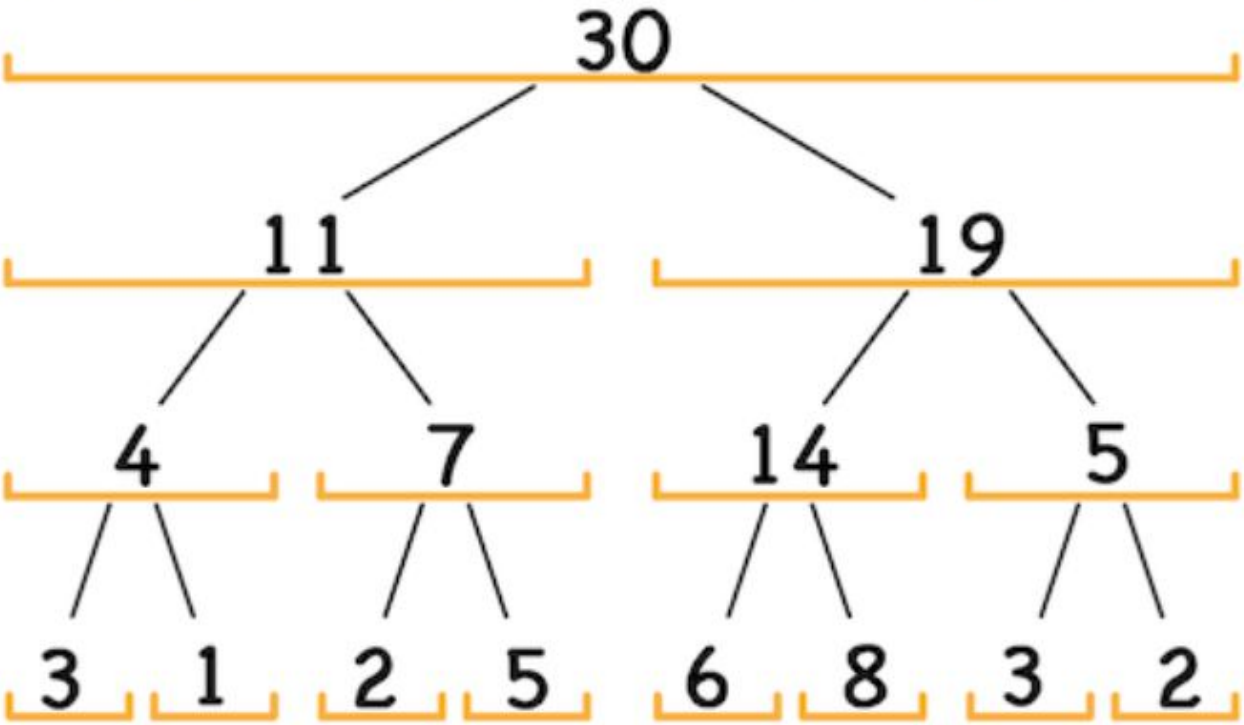


Segment Tree - Actualización

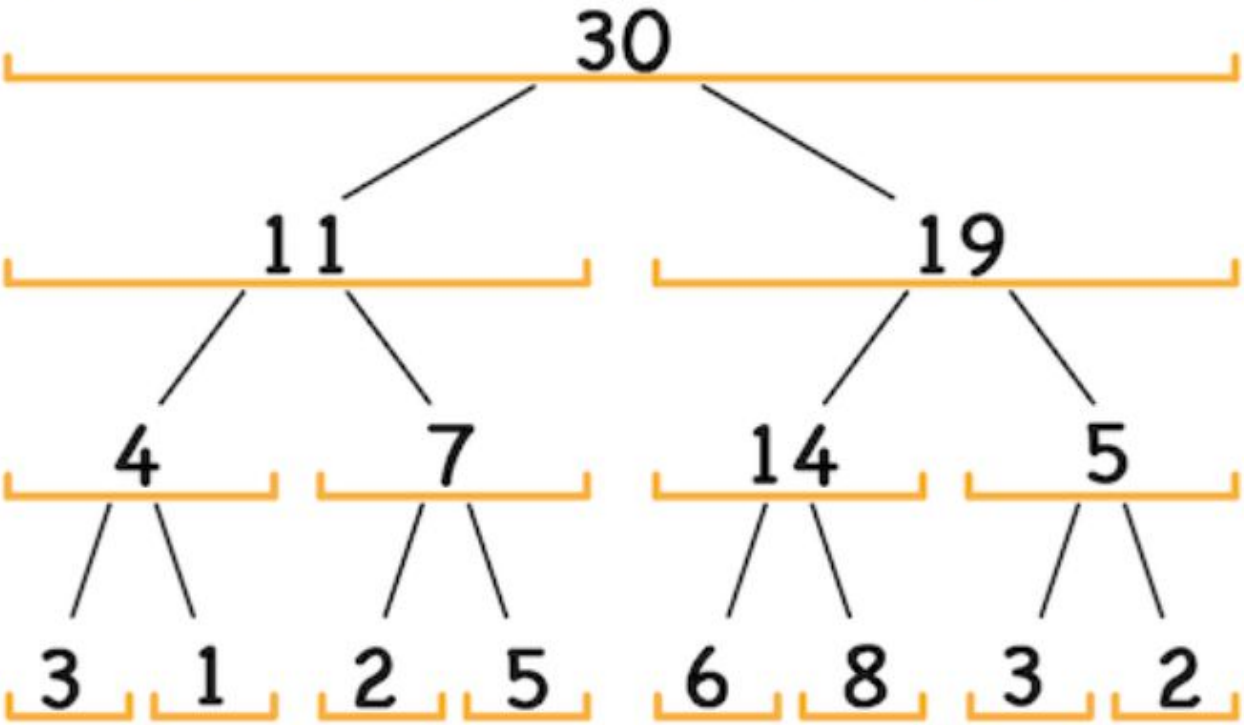


Actualizado 4 por 8

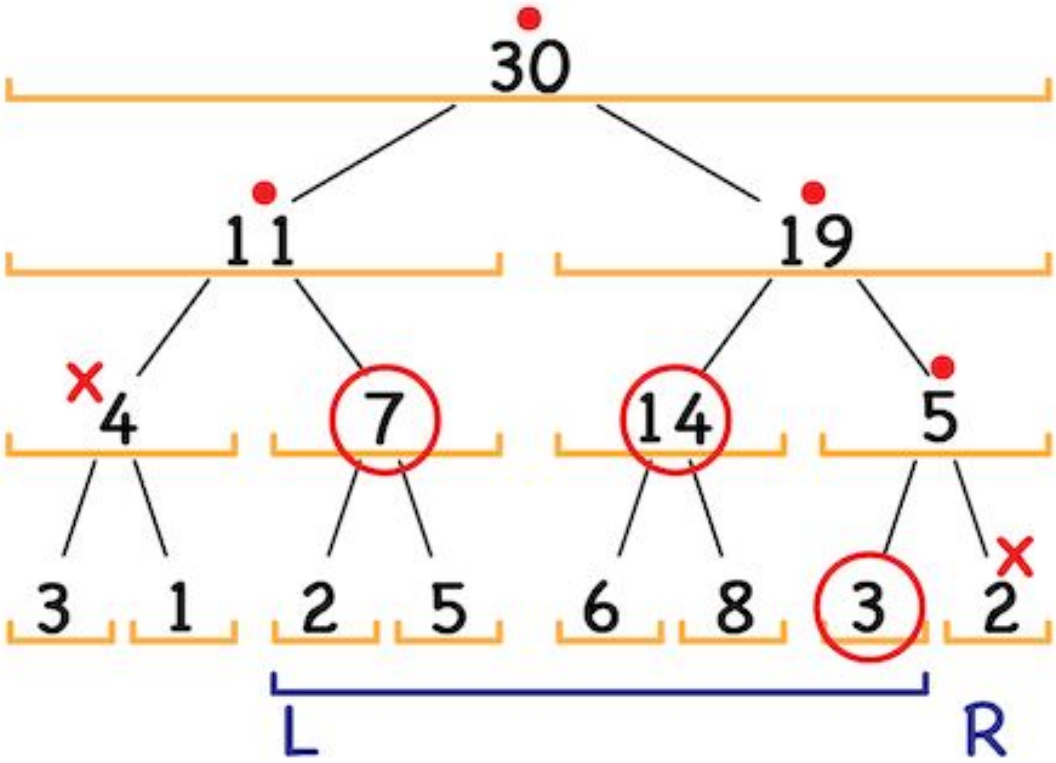
Segment Tree - Query



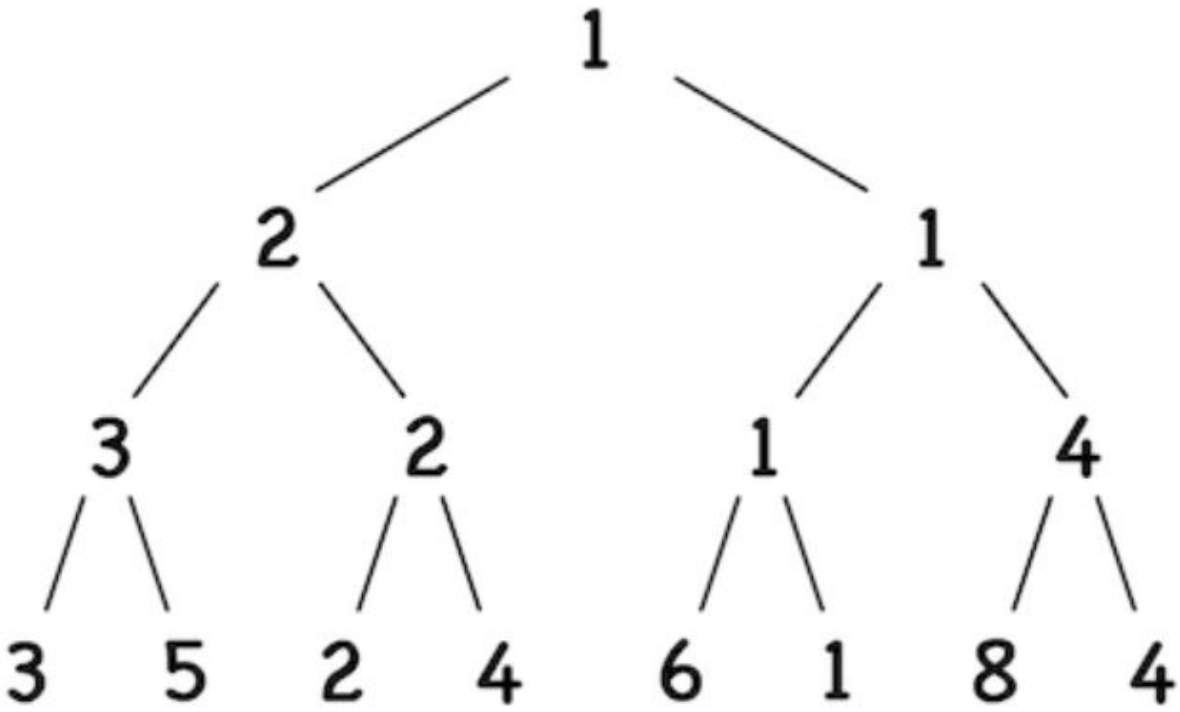
Segment Tree - Query



Segment Tree - Query



Min Segment Tree



Segment Tree Código

https://github.com/nicolasalba/competitive/blob/main/cpp/7.%20query/Min%20Segment%20Tree.cpp

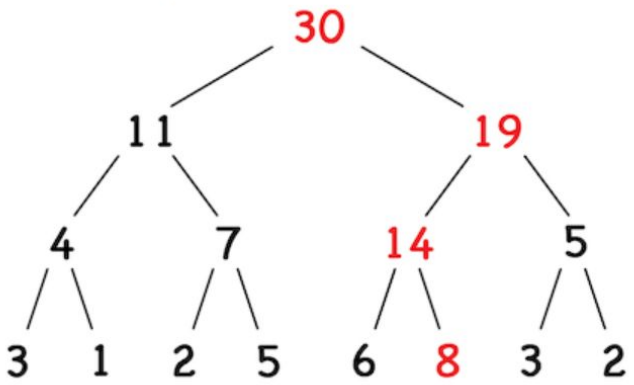
https://cp-algorithms.com/data_structures/segment_tree.html

Segment Tree Código

```
void update(int i, int sl, int sr, int pos, ll diff) {
    if (sl <= pos && pos <= sr) {
        if (sl == sr) {
            tree[i] += diff;
        } else {
            int mid = (sl + sr) / 2;
            update(i * 2 + 1, sl, mid, pos, diff);
            update(i * 2 + 2, mid + 1, sr, pos, diff);
            tree[i] = max(tree[i * 2 + 1], tree[i * 2 + 2]);
        }
    }
}
```

Segment Tree Código

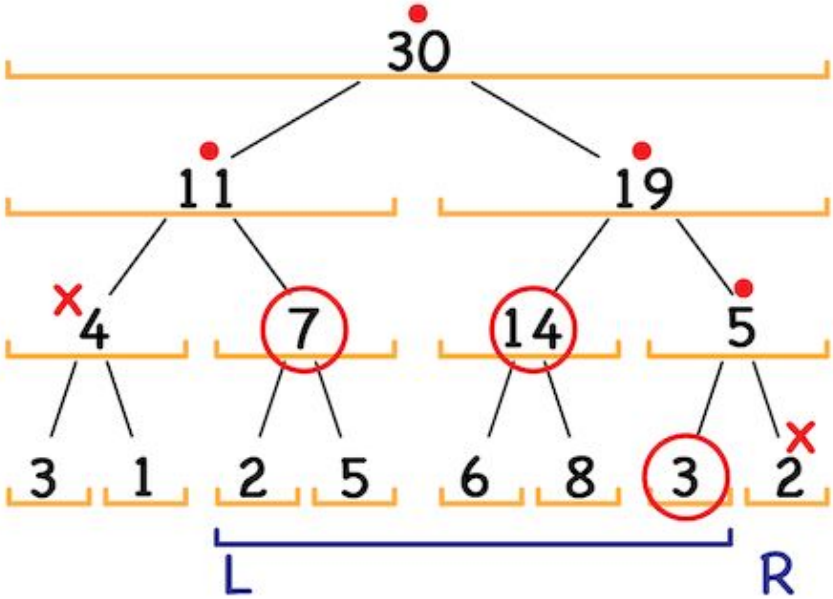
```
void update(int i, int sl, int sr, int pos, ll diff) {
    if (sl <= pos && pos <= sr) {
        if (sl == sr) {
            tree[i] += diff;
        } else {
            int mid = (sl + sr) / 2;
            update(i * 2 + 1, sl, mid, pos, diff);
            update(i * 2 + 2, mid + 1, sr, pos, diff);
            tree[i] = max(tree[i * 2 + 1], tree[i * 2 + 2]);
        }
    }
}
```




```
void update(int pos, ll diff) {
```

Segment Tree Código

```
ll query(int i, int sl, int sr, int l, int r) {
    if (l <= sl && sr <= r) {
        return tree[i];
    } else if(sr < l || r < sl) {
        return INT64_MIN;
    } else {
        int mid = (sl + sr) / 2;
        auto a = query(i * 2 + 1, sl, mid, l, r);
        auto b = query(i * 2 + 2, mid + 1, sr, l, r);
        return max(a, b);
    }
}
```




```
ll query(int l, int r) {
    return query(0, 0, n - 1, l, r);
```

Tiempo de Complejidad

Update: log(n)

Query: log(n)

Cuando usar Prefix Sum, Fenwick, Segment Tree?

Prefix Sum: Solo queries sin update

BIT: Queries con Update de Sumas

Segment Tree: Queries y updates con operaciones y estructuras más complejas

Problema

Finding the maximum and the number of times it appears

Problema

```cpp
void update(int i, int sl, int sr, int pos, ll diff) {
    if (sl <= pos && pos <= sr) {
        if (sl == sr) {
            tree[i].F += diff;
        } else {
            int mid = (sl + sr) / 2;
            update(i * 2 + 1, sl, mid, pos, diff);
            update(i * 2 + 2, mid + 1, sr, pos, diff);
            tree[i] = max(tree[i * 2 + 1], tree[i * 2 + 2]);
            if (tree[i * 2 + 1].F == tree[i * 2 + 2].F) {
                tree[i].S = tree[i * 2 + 1].S + tree[i * 2 + 2].S;
            }
    }
}
```

```cpp
pair<ll,ll> query(int i, int sl, int sr, int l, int r) {
    if (l <= sl && sr <= r) {
        return tree[i];
    } else if(sr < l || r < sl) {
        return {INT64_MIN,0};
    } else {
        int mid = (sl + sr) / 2;
        auto a = query(i * 2 + 1, sl, mid, l, r);
        auto b = query(i * 2 + 2, mid + 1, sr, l, r);
        if (a.F == b.F) {
            return {a.F,a.S+b.S};
        }
        return max(a,b);
    }
}
```

General Segment Tree

$$a=\{a1,a2,a3,a4,a5,a6,\ldots,a_n\}$$

$$query(l,r) = a[l]\oplus a[l+1]\oplus\ldots\oplus a[r]$$

General Segment Tree

$$query(l,r) = a[l] \oplus a[l+1] \oplus \ldots \oplus a[r]$$

Estructuras Algebraicas

Monoide

Estructura Algebraica Monoide

Elemento x –> (1,2,3)

Operación Binaria –> $\oplus$

Elemento Neutro –> e

Propiedad Asociativa

Estructura Algebraica Monoide

Elemento x –> (1,2)

Operación Binaria –> Tupla$\oplus$Tupla=Tupla

$$(a,b)\oplus(c,d) = (a+c,b+d)$$

Elemento Neutro –> (0,0)

$$(a,b)\oplus(0,0) =(a+0,b+0)=(a,b)$$

Propiedad Asociativa

$$((a,b)\oplus(c,d))\oplus(e,f)=(a,b)\oplus((c,d)\oplus(e,f))$$

$$= (a+c+e,b+d+f)$$

General Segment Tree

```
// >>>>>>>>>> Implement
struct Node { ll x = 0; };

Node e() { return Node(); }

Node op(Node &a, Node &b) {
    Node c;
    c.x = a.x + b.x;
    return c;
}
```

Codigo General Segment Tree

https://github.com/nicolasalba/competitive/blob/main/cpp/7.%20query/general_segtree.cpp

Problemas

Dado un arreglo a de n números enteros, y q queries de dos tipos:

1. Actualizar a[i] = k
2. Encontrar la máxima suma de un subarray del segmento L, R

$(1 \leq n \leq 10^5)$. $(1 \leq q \leq 10^5)$.

Respuesta

Dado un arreglo a de n números enteros, y q queries de dos tipos:

1. Actualizar a[i] = k
2. Máxima suma de un subarray de subarray L, R

```
// >>>>>>>>> Implement
struct Node { ll pref, suf, val, sum; };

Node e() {
    return {0, 0, 0, 0};
}

Node op(Node &a, Node &b) {
    Node c;
    c.pref = max(a.pref, a.sum + b.pref);
    c.suf = max(b.suf, b.sum + a.suf);
    c.val = max({a.val, b.val, a.suf + b.pref});
    c.sum = a.sum + b.sum;
    return c;
}
// <<<<<<<
```

Problema

The function $f(x)$ is defined as the sum of the digits in base 10 of x minus the smallest digit. For example, $f(324) = 3 + 2 + 4 - 2 = 7$ or $f(1100) = 1 + 1 + 0 + 0 - 0 = 2$.

Given an array a of n integers numbered from 1 to n, process q queries of two types:

1. Given two integers x and v, replace the value of a_x with v
2. Given two integers l and r, calculate the minimum number of times the function $f(a_i)$ must be applied to the elements of a in the range $[l, r]$, so that all the elements in that range become equal.

Note that, while processing type 2 queries, the elements of a do not change.

$(1 \leq n \leq 10^5)$. $(0 \leq a_i \leq 10^{18})$. $(1 \leq q \leq 10^5)$.

Problema Sansi Cup

input

```
4
50 5 15 4
3
2 1 3
1 3 14
2 2 4
```

output

```
2
4
```

- The first query is of type 2, the function $f(a_i)$ needs to be applied twice for all the elements in the range $[1, 3]$ to become equal: $f(50) = 5, f(15) = 5$.
- The second query is of type 1, the value of a_3 is replaced by 14.
- The third query is of type 2, the function $f(a_i)$ needs to be applied four times for all the elements in the range $[2, 4]$ to become equal: $f(5) = 0, f(f(14)) = 0, f(4) = 0$.

Problema Sansí Cup

$$query(l,r) = a[l] \oplus a[l+1] \oplus \ldots \oplus a[r]$$

a[i] = Mapa de costos

(e.g a[i][2] costo de convertir a[i] en 2)

$a[i] \oplus a[j]$ = Mapa de Costos combinado de a[i] y a[j]

Lazy Propagation

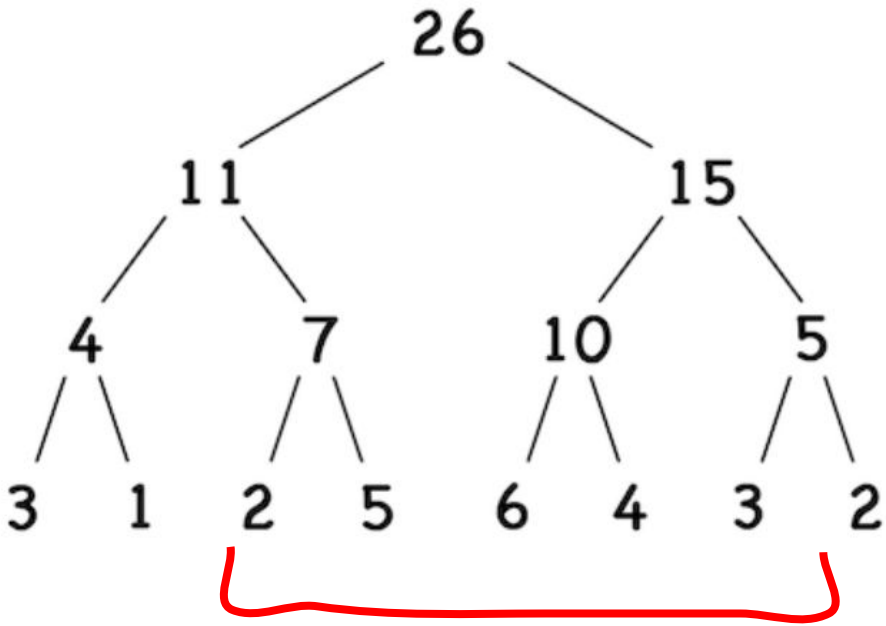
Dado un array ‘N’ y ‘Q’ operaciones de tipos:

1. Dado ‘i’ y ‘k’, actualizar a[i] = k
2. Dado ‘L, ‘R’, y ‘k’, actualizar a[i] += k, L <= i <= R
3. Dado ‘L’ y ‘R’, responder a[L] + … + a[R]

Lazy Propagation Codigo

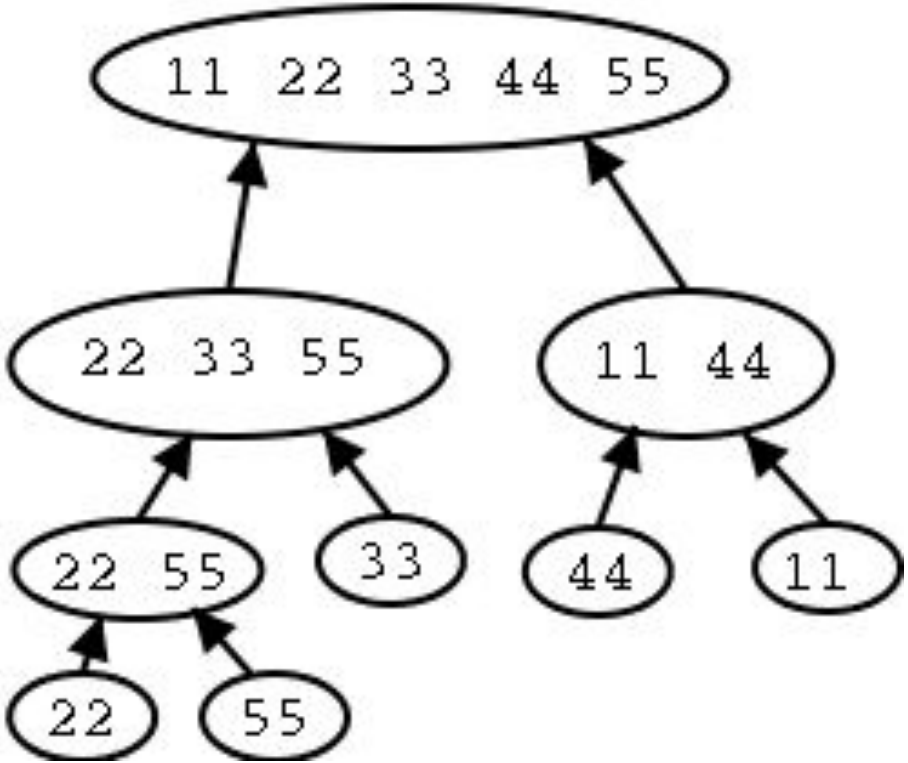
https://github.com/nicolasalba/competitive/blob/main/cpp/7.%20query/struct%20lazy%20tree.cpp

Lazy Propagation



Sumar 4 o aplicar f(x) = x + 4

Merge Sort Tree



Thanks

https://codeforces.com/profile/nicolasalba

CREDITS: This presentation template was created by **Slidesgo,** including icons by **Flaticon,** infographics & images by **Freepik**

Please, keep this slide for the attribution